itthynk

SOFTWARE DEVELOPMENT INTERNSHIP PROGRAMME · COHORT 2026

Demo moved · now Monday 14 Sep, not Friday

Helpdesk System — Week 4 Feedback

The individual pieces here — hashing, the register endpoint, the login endpoint — are each written reasonably well. The pattern worth talking through is what happens between the pieces: the interface was built before it was connected to anything real, and the one file whose job was to say "no" was never written at all.

Repo · `cohort-2026/helpdesk-system` **Sprint work** · Authentication & Authorization, Mon 7 – Fri 11 Sep 2026

Demo · Monday 14 Sep 2026 (moved from Friday)

WHAT THIS DOCUMENT IS — AND ISN'T

This is **feedback, not a fix-it list.** Every item below is here to help you see where your own reasoning went wrong and think through a better way to approach it — not to hand you code to paste in. Each one points at the real file and line in your repo so you can go look at it yourselves, and some show the actual code that's there right now — not because copying a replacement is the point, but because seeing your own code named directly is more useful than a description of it. The short examples further down are illustrative, to make an idea concrete, not to be copied.

What to demo on Monday

The brief's own sequence, walked through against what this repo actually does today — from the same audit this feedback is drawn from.

Before you present: `main` has no auth code at all — check out `development` before doing anything else. `feature/auth-login-session` adds logout/me but never routes them, so don't rely on it without wiring the routes in first.

Step 1 Register a new account through the sign-up form

No UI yet

ON SCREEN

Nothing happens against the real backend — there's no sign-up form in the frontend yet.

IN THE CODE / TERMINAL

`POST /api/auth/register` works fine from curl/Postman: 201, a bcrypt hash lands in the DB, no password comes back in the response.

Demo registration via curl instead of the browser for now — see Solution 1 to wire up the real form.

Step 2 Log in with that account

Currently fake

ON SCREEN

`SignInPage.tsx` maps a few hardcoded emails to roles and never calls the backend — whatever you type in, it signs you in without checking anything.

IN THE CODE / TERMINAL

Network tab: no request to `/api/auth/login` is ever sent.

This will look like it works and isn't real. Fix the form before presenting — see Solution 2.

Step 3 Refresh the page and confirm you're still signed in

No cookie

ON SCREEN

Depends entirely on Step 2 being fixed first.

IN THE CODE / TERMINAL

Even once wired up: the token comes back in the JSON body, not a cookie — DevTools → Application → Cookies will show nothing set.

See Solution 3 — this is one of the two non-negotiable rules currently failing.

Step 4 Visit a protected page while signed out

UI only

ON SCREEN

`ProtectedRoute.tsx` correctly redirects you to `/login`.

IN THE CODE / TERMINAL

But hitting the underlying API route directly (curl, no cookie) still succeeds — `authMiddlware.ts` is empty.

The redirect is convenience, not security, until Solution 4 lands.

Step 5 Log in as a normal user, try an admin-only action

Not built

ON SCREEN

Nothing to show — no admin-only page exists yet.

IN THE CODE / TERMINAL

No `requireRole` middleware exists anywhere in the repo.

Step 6 Log out

Unrouted

ON SCREEN

The logout button won't reach the backend even if it's wired to `AuthContext`.

IN THE CODE / TERMINAL

The `/logout` controller exists but is never mounted on a route — Network tab shows 404.

Step 7 Try a wrong password, then a non-existent email

Enumération bug

ON SCREEN

The two errors look and read differently.

IN THE CODE / TERMINAL

404 `"User not found"` vs 401 `"Invalid password"` — different status codes too.

This is exactly the enumeration bug the brief warns about. See Solution 5.

RECOMMENDED ORDER TO WORK THROUGH THIS

WHEN	ISSUE	WHERE
1st	Wire in the empty auth middleware	Lesson 2
1st	Get CORS configured so the frontend can reach the backend at all	Lesson 8
2nd	Move the session into a cookie, and connect the real forms to it	Lessons 1 & 3
3rd	Fix the mismatched duplicate-email response, and add the missing validation	Lessons 4 & 7
4th	Commit the schema, remove the fallback secret, add a .env.example	Lessons 5 & 9
anytime	Reconcile the stale branch, the mismatched dependency, and the misleading branch names	Lessons 6 & 10

Nothing else is worth polishing until requests are actually being checked and can actually reach the server — those two come first, everything else can happen in parallel.

Where this went wrong, and how to think about it differently

In the order it'll matter most — not necessarily the order you'll want to tackle it.

1 The interface was built before the connection to it was real

`Frontend/src/pages/Auth pages/SignInPage.tsx · no register page exists in any branch`

WHAT WENT WRONG

> The sign-in screen doesn't check anything against the server — it looks at the email you typed and decides what to pretend, without ever sending a request. It was built to look finished before it was wired up to something that actually works.

WHAT'S ACTUALLY THERE RIGHT NOW

Confirmed by a repo-wide search, not just this file: no `fetch` or `axios` call exists anywhere in the frontend, and no register page exists at all. SignInPage.tsx maps a handful of hardcoded emails to roles and never contacts the backend.

WHY IT MATTERS

> *It's like fitting a keypad to a door that was never wired to the lock. From the outside it looks like security — there's a keypad, it lights up, it beeps. But the door was never actually locked, so it doesn't matter what anyone types.*

A BETTER WAY TO THINK ABOUT IT

> Get the real request working first, however ugly — a plain form, or even just curl — before spending time making the screen look good. Once you can watch a real 200 or a real 401 come back from your own server, you know there's something underneath to make pretty. Polishing the outside first means you can ship something that looks done and isn't.

EXAMPLE

illustrative only — the point is the order you build in, not this exact code

```
// build and prove this first, even ugly:
const res = await fetch("/api/auth/login", {
  method: "POST",
  credentials: "include",
  body: JSON.stringify({ email, password }),
});
console.log(res.status); // watch a real 401 happen before you style anything

// only once that's real, make the form nice to look at
```

2 The one file whose job was to say "no" was left empty

`Backend/src/middleware/authMiddlware.ts`

WHAT WENT WRONG

> There's a file dedicated entirely to deciding whether a request is allowed to continue — and it does nothing. The backend currently trusts every request that arrives, regardless of who sent it.

WHAT'S ACTUALLY THERE RIGHT NOW

This file exists but contains zero lines of code — confirmed empty — and it's never imported anywhere else in the project. There is currently no server-side check on any route at all.

WHY IT MATTERS

> *It's like hiring a security guard, giving them a desk, a uniform, and a badge — and never actually telling them to check anyone's ID. The desk is there. It looks staffed. Nobody's checking anything.*

A BETTER WAY TO THINK ABOUT IT

> Write the one function that answers "is this person allowed to be here — yes or no" before building anything that assumes the answer is already being checked. Then prove it: remove your session cookie and confirm a protected route actually refuses you. Don't take it on faith that a file existing means it's doing its job.

EXAMPLE

illustrative only — the pattern matters more than the exact code

```
export function requireAuth(req, res, next) {
  const token = req.cookies?.session;
  if (!token) return res.status(401).json({ message: "Not signed in" });
  // ...verify it, attach the user, then call next()
}
// then: remove your cookie in devtools and confirm the route actually refuses you
```

3 The visitor was handed their own pass instead of it staying at the door

`Backend/src/controllers/authController.ts:55`

WHAT WENT WRONG

> After logging in, the session token is handed straight back to the browser in the response, for the frontend to store and carry around itself. That means any script running on the page — including one from a compromised dependency — can pick it up and read it.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, from the audit notes

```
res.status(200).json({ token, user... })
// (as recorded in the audit — the "..." marks additional fields, not literal
code)
// no res.cookie( call exists anywhere in this repo
```

WHY IT MATTERS

> *A sealed courier envelope only the recipient can open is one thing. Reading the contents out loud in a waiting room, so anyone nearby can note them down, is another. Both "deliver" the message — only one keeps it private.*

A BETTER WAY TO THINK ABOUT IT

> Let the browser hold the session itself, in a cookie marked so JavaScript can't read it (httpOnly). The server sets it once; the browser sends it back automatically on every request after that. The frontend never needs to touch, store, or think about it at all — which is exactly what makes it safer.

EXAMPLE

illustrative only

```
res.cookie("session", token, {
  httpOnly: true,   // JavaScript on the page cannot read this, even if it tries
  sameSite: "lax",
  secure: process.env.NODE_ENV === "production",
});
// nothing goes in the response body — the browser already has it now
```

4 Two different answers gave away more than either one alone would

`Backend/src/controllers/authController.ts:44 & :51`

WHAT WENT WRONG

> A wrong email currently gets a different message and a different status code than a wrong password. Neither answer alone reveals anything sensitive — but together, they let someone work out which emails are actually registered, one guess at a time.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
// line 44 — wrong email:
res.status(404).json({ message: "User not found" });

// line 51 — wrong password:
res.status(401).json({ message: "Invalid password" });
```

WHY IT MATTERS

> *Imagine a receptionist who says "you're not on our list" to a stranger, but "that's not the right password" to someone whose name they recognise. Neither sentence seems harmful by itself. But a stranger trying every name in a phone book would quickly learn exactly who has an account here, just from which sentence they get back.*

A BETTER WAY TO THINK ABOUT IT

> Make both failure paths produce the identical response — same message, same status code — so there's genuinely no way to tell, from the outside, which part was wrong. It should feel almost unhelpful to a real user who mistyped their email. That's the point.

EXAMPLE

illustrative only

```
if (!user || !passwordMatches) {
  return res.status(401).json({ message: "Invalid email or password" });
}
// this line now runs whichever part was actually wrong — the caller can't tell
which
```

5 A database that only exists in someone's memory

`checked across all 15 branches in the repo`

WHAT WENT WRONG

> There's no file anywhere in the project that says what the `users` table actually looks like. If it exists, it was created by clicking around in a dashboard — which means nobody else on the team can see it, question it, or rebuild it if it's lost.

WHAT'S ACTUALLY THERE RIGHT NOW

No `.sql` file, migration, or schema definition exists anywhere in the project's git history, on any branch — confirmed by searching the full repo, not just the current one.

WHY IT MATTERS

> *It's the difference between a recipe written down and a dish only one chef knows how to make from memory. If that chef is away, or misremembers a step, nobody else can check whether tonight's version is even the same dish.*

A BETTER WAY TO THINK ABOUT IT

> Write the table definition as a `.sql` file and commit it, the same as any other code. It becomes something the whole team can read, review, and re-run to get the exact same database anyone else has — instead of a shared assumption nobody wrote down.

EXAMPLE

illustrative only

```
CREATE TABLE users (
  id            SERIAL PRIMARY KEY,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role          TEXT NOT NULL DEFAULT 'user',
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
-- commit this file. now anyone can read exactly what the table is supposed to be.
```

6 Two branches now disagree about whether a duplicate email is allowed

`development branch: fixed, returns 409 · feature/auth-login-session: still returns 400 (predates the fix)`

WHAT WENT WRONG

> One branch of the project correctly rejects a duplicate email with a 409. Another branch — still open, not yet merged — still has the older bug where it just returns a plain 400 instead. The fix exists; it just didn't reach every place the same code lives.

WHAT'S ACTUALLY THERE RIGHT NOW

The duplicate-email fix landed on `development`. The still-open `feature/auth-login-session` branch was created before that fix and still has the old behaviour.

WHY IT MATTERS

> *It's like fixing a typo in a printed handout, but the version still sitting in the photocopier tray never gets swapped out — some people walk away with the corrected page, others with the old one, and nobody handing them out can tell which is which anymore.*

A BETTER WAY TO THINK ABOUT IT

> When a fix lands on one branch, it's worth checking whether the same bug exists on any other branch that's still active — especially ones that split off before the fix happened. A quick search for the same pattern elsewhere costs a minute; finding out later that two branches disagree costs much more.

EXAMPLE

illustrative only — what the two branches currently disagree about

```
// development (fixed):
if (existingUser) return res.status(409).json({ message: "Email already
registered" });

// feature/auth-login-session (predates the fix, still open):
if (existingUser) return res.status(400).json({ message: "Something went wrong"
});
// same bug, still live on a branch that hasn't been rebased onto the fix
```

7 The form checks almost nothing before sending it to the server

`Backend/src/controllers/authController.ts:8-10`

WHAT WENT WRONG

> The registration endpoint currently only checks that fields aren't missing — there's no check on email format, and no minimum password length enforced on the server. Someone could register with "a" as a password, or "not-an-email" as the email, and it would succeed.

WHAT'S ACTUALLY THERE RIGHT NOW

Validation here only checks that fields are present (`!name || !surname || ...`) — there's no email-format regex and no minimum password length enforced.

WHY IT MATTERS

> *It's like a bouncer who checks that you're holding some kind of card, without ever looking at whether it's actually a valid ID — any card-shaped object gets you in.*

A BETTER WAY TO THINK ABOUT IT

> Every rule the brief asks for (minimum password length, a valid email shape) needs to be enforced exactly where the data is trusted — on the server, before it's used — not assumed because the frontend form happens to have a matching input type.

EXAMPLE

illustrative only

```
if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {
  return res.status(400).json({ message: "Invalid email format" });
}
if (password.length < 8) {
  return res.status(400).json({ message: "Password must be at least 8 characters"
});
}
```

8

The frontend can't reach the backend across origins yet, because nobody's told it to try

`Backend/index.ts · no cors package in package.json`

WHAT WENT WRONG

> There's no cors package installed anywhere in this project, and no cors() call in the backend at all. Even once the login form is wired up for real, a request from the frontend's address to the backend's address will be blocked by the browser by default.

WHAT'S ACTUALLY THERE RIGHT NOW

There is no `cors` dependency installed and no `cors(...)` call anywhere in the backend — confirmed by checking the package file and searching the code.

WHY IT MATTERS

> *It's like building a phone system where nobody's agreed which country codes are allowed to call in — until that's set up, every call gets rejected, no matter how correctly it was dialled.*

A BETTER WAY TO THINK ABOUT IT

> This is worth setting up early and deliberately — a specific allowed origin, paired with credentials handling — rather than discovering it only once the login form is finally sending real requests and everything mysteriously fails.

EXAMPLE

illustrative only

```
// npm install cors, then in Backend/index.ts:
import cors from "cors";
app.use(cors({ origin: "http://localhost:5173", credentials: true }));
// this has to be paired with credentials: "include" on the frontend's fetch calls
(Lesson 1)
```

9

A backup password that's easy to find, and a project with no list of what it needs to run

`no .env.example anywhere in the repo · JWT secret falls back to the literal word "secret" if unset`

WHAT WENT WRONG

> If the real secret used to sign sessions isn't set, the code quietly falls back to the literal word "secret". And there's no .env.example file anywhere, so a new team member has no list of what environment variables the project actually needs.

WHAT'S ACTUALLY THERE RIGHT NOW

If `JWT_SECRET` isn't set, both the login and the unmerged `/me` handler fall back to the hardcoded string `"secret"`. No `.env.example` file exists in any branch.

WHY IT MATTERS

> *It's the spare key under the mat again, except this time the mat itself has no label saying "there's supposed to be a key here at all" — a new person joining has no way to even know what they're meant to set up.*

A BETTER WAY TO THINK ABOUT IT

> Two small, cheap habits fix both: crash loudly if a required secret is missing rather than substituting a guessable one, and commit an .env.example with the variable names present but the values blank, so anyone starting fresh knows exactly what to fill in.

EXAMPLE

illustrative only

```
// instead of falling back silently:
const secret = process.env.JWT_SECRET ?? "secret";

// fail loudly instead:
const secret = process.env.JWT_SECRET;
if (!secret) throw new Error("JWT_SECRET is not set");

// and commit a .env.example alongside it:
// JWT_SECRET=
// DATABASE_URL=
```

10

Two libraries doing the same job, and branch names that don't tell the truth

`Backend/package.json declares bcrypt, but authController.ts imports bcryptjs · branches 22-featureauth-frontend and feat/logout`

WHAT WENT WRONG

> The project's dependency list says one hashing library is installed, but the actual code imports a different one — it only works today because of a coincidental folder outside the project that happens to have it installed. Separately, two branch names promise things they don't contain.

WHAT'S ACTUALLY THERE RIGHT NOW

`bcryptjs` and `jsonwebtoken` are imported in the code but not listed in `package.json` or the lockfile — they only resolve because of a stray root-level `node_modules` outside the project. Separately, `22-featureauth-frontend` contains no auth work (layout only), and `feat/logout` has nothing to do with logout.

WHY IT MATTERS

> *It's like a filing cabinet where the drawer labelled "invoices" is where someone actually put the meeting notes — technically findable if you already know not to trust the label, but useless to anyone else relying on it.*

A BETTER WAY TO THINK ABOUT IT

> Keep the declared dependency and the imported one in sync (install the one actually being used). Treat a branch name as a promise to whoever reads it next, including a future you — if a branch doesn't turn out to be what its name says, it's worth renaming it or noting that clearly.

EXAMPLE

illustrative only

```
// authController.ts imports this:
import bcrypt from "bcryptjs";
// but package.json declares this instead:
"dependencies": { "bcrypt": "^5.1.0" }
// fix: npm install bcryptjs --save (or switch the import to match what's
declared)
```

Checklist — work through this before Monday

One box per lesson above, phrased as something you can actually check off. Tick it once you can point to where it's true in your own code — not once you've read the explanation. Items marked **Critical** are where the audit found a real gap; be ready to show it live, not describe it.

- [ ] **Do the sign-in and register forms actually call your backend and handle a real error response?** CRITICAL
- [ ] **Does authMiddlware.ts actually reject a request with no cookie, on every route that needs it?** CRITICAL
- [ ] **Is the session set as an httpOnly cookie, with nothing sensitive in the response body?** CRITICAL
- [ ] **Does a wrong email and a wrong password now return the exact same message and status code?** CRITICAL
- [ ] **Is the users table committed as a .sql file anyone on the team can read and re-run?** CRITICAL
- [ ] Does every open branch return 409 on a duplicate email, not just the merged one?
- [ ] Does registration reject a bad email format and a too-short password, not just an empty field?
- [ ] **Is a cors() call, with a specific origin, actually present in the backend?** CRITICAL
- [ ] Does the app refuse to start without a real JWT secret, and is there a .env.example listing what's needed?
- [ ] Does the code import the same hashing library that's actually declared in package.json? Do branch names describe what they actually contain?